



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martínez

Asignatura: Estructura de datos y algoritmos I

Grupo: 5

No de Práctica(s):

Integrante(s): Villarruel Sanchez David Uriel

No. de lista o brigada: 40

Semestre: 2026-2

Fecha de entrega: 17/05/2026

Observaciones:

CALIFICACIÓN: _____

Índice:

Índice:.....	2
objetivos:.....	2
Introducción:.....	2
Desarrollo.....	3
Conclusión:.....	10

Objetivos

El sistema deberá:

- Leer variables desde un archivo.
- Leer una expresión infija.
- Convertir la expresión a notación postfija.
- Evaluar la expresión postfija.
- Mostrar el resultado final.

Introducción

Este proyecto trata sobre un programa hecho en lenguaje C que puede leer y resolver expresiones matemáticas desde un archivo de texto, usando variables y operaciones como suma, resta, multiplicación, división y potencias. La idea principal del programa es tomar una expresión escrita de forma normal y convertirla a una expresión postfija, ya que así es más fácil resolverla respetando el orden correcto de las operaciones sin que haya errores con prioridades o paréntesis.

Para que el programa funcione se utilizaron estructuras como pilas y colas, las cuales ayudan a organizar mejor la información mientras se procesa la expresión matemática. También se trabajó con archivos de texto para leer automáticamente las variables junto con sus valores y la operación que se quiere resolver. Además, el proyecto se dividió en diferentes archivos para mantener el código más ordenado, haciendo que cada parte tenga una función específica y sea más fácil entender cómo funciona todo el programa.

Desarrollo

cola.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/cola.h"
6
7  Cola* crearCola() {
8      Cola *c = (Cola *)malloc(sizeof(Cola));
9      if (c == NULL) {
10         printf("Error: sin memoria para crear cola.\n");
11         return NULL;
12     }
13     c->frente = NULL;
14     c->final = NULL;
15     return c;
16 }
17
18 // Asigna memoria al puntero de la cola c
19
20 int colaVacia(Cola *c) {
21     return c->frente == NULL;
22 }
23
24 // Revisa si el frente de la pila apunta a NULL
25
26 void enqueue(Cola *c, void *dato, size_t size) {
27     NodoCola *nuevo = (NodoCola *)malloc(sizeof(NodoCola));
28     if (nuevo == NULL) {
29         printf("Error: sin memoria para enqueue.\n");
30         return;
31     }
32     nuevo->dato = malloc(size);
33     if (nuevo->dato == NULL) {
34         printf("Error: sin memoria para dato en enqueue.\n");
35         return;
36     }
37     memcpy(nuevo->dato, dato, size);
38     nuevo->siguiente = NULL;
39
40     if (colaVacia(c)) {
41         c->frente = nuevo;
42         c->final = nuevo;
```

```
43     } else {
44         c->final->siguiente = nuevo;
45         c->final = nuevo;
46     }
47 }
48
49 char dequeue(Cola *c) {
50     if (colaVacia(c)) {
51         printf("Error: dequeue en cola vacia.\n");
52         return '\0';
53     }
54     NodoCola *temp = c->frente;
55     c->frente = temp->siguiente;
56     if (c->frente == NULL)
57         c->final = NULL;
58
59     char dato = *((char *)temp->dato);
60     free(temp->dato);
61     free(temp);
62     return dato;
63 }
64
65 void imprimirCola(Cola *c) {
66     NodoCola *actual = c->frente;
67     while (actual != NULL) {
68         printf("%c", *((char *)actual->dato));
69         if (actual->siguiente != NULL)
70             printf(" ");
71         actual = actual->siguiente;
72     }
73     printf("\n");
74 }
75
76 void destruirCola(Cola *c) {
77     while (!colaVacia(c)) {
78         NodoCola *temp = c->frente;
79         c->frente = temp->siguiente;
80         free(temp->dato);
81         free(temp);
82     }
83     free(c);
```

Este archivo es prácticamente el que pone a funcionar todo el programa. Aquí es donde se cargan las variables y la expresión matemática desde un archivo de texto para después empezar con el proceso. Primero se muestra la expresión tal y como viene escrita para asegurarse de que se leyó correctamente.

Después de eso, el programa convierte la expresión a postfija y también la imprime en pantalla para poder revisar cómo quedó. Ya con la expresión convertida, se hace la operación usando los valores de las variables y al final se muestra el resultado. También se elimina la memoria usada para evitar errores o datos guardados sin necesidad. En pocas palabras, este archivo es el que coordina todo lo que pasa en el programa.

evaluator.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <math.h>
4
5  #include "../include/evaluator.h"
6  #include "../include/pila.h"
7  #include "../include/cola.h"
8
9  float obtenerValorVariable(
10     Variable variables[],
11     int total,
12     char nombre
13 ) {
14     for (int i = 0; i < total; i++) {
15         if (variables[i].nombre == nombre) {
16             return variables[i].valor;
17         }
18     }
19     printf("Error mi chav@ la variable '%c' no esta definida.\n", nombre);
20     return 0.0;
21 }
22
23 float evaluarPostfija(
24     Cola* postfija,
25     Variable variables[],
26     int totalVariables
27 ) {
28     Pila* pilaNum = crearPila();
29
30     while (colaVacia(postfija) == 0) {
31         char actual = dequeue(postfija);
32
33         if ((actual >= 'a' && actual <= 'z') || (actual >= 'A' && actual <= 'Z')) {
34             float valor = obtenerValorVariable(variables, totalVariables, actual);
35             push(pilaNum, &valor, sizeof(float));
36         }
37         else if (actual == '+' || actual == '-' || actual == '*' || actual == '/' || actual == '^') {
38
39             float b = 0.0;
40             if (pilaVacia(pilaNum) == 0) {
41                 b = ((float)pilaNum->tope->dato);
42                 pop(pilaNum);
43             }
```

```

44
45     float a = 0.0;
46     if (pilaVacía(pilaNum) == 0) {
47         a = ((float)pilaNum->tope->dato);
48         pop(pilaNum);
49     }
50
51     float res = 0.0;
52
53     switch (actual) {
54         case '+': res = a + b; break;
55         case '-': res = a - b; break;
56         case '*': res = a * b; break;
57         case '/':
58             if (b == 0.0) {
59                 printf("Como vas a dividir entre 0??? :c.\n");
60                 destruirPila(pilaNum);
61                 return 0.0;
62             }
63             res = a / b;
64             break;
65         case '^': res = (float)pow((double)a, (double)b); break;
66     }
67
68     push(pilaNum, &res, sizeof(float));
69 }
70
71
72 float fin = 0.0;
73 if (pilaVacía(pilaNum) == 0) {
74     fin = ((float)pilaNum->tope->dato);
75     pop(pilaNum);
76 } else {
77     printf("Error: expresion vacia o invalida.\n");
78 }
79
80 destruirPila(pilaNum);
81 return fin;
82 }

```

Este archivo es el que se encarga de sacar el resultado final de la operación. Aquí el programa toma la expresión postfija y empieza a resolverla usando los valores de las variables que se leyeron del archivo.

Primero busca cuánto vale cada variable según la letra que aparezca. Después, mientras recorre la expresión, si encuentra una variable la guarda en una pila, toma los últimos valores guardados, hace la operación y vuelve a meter el resultado a la pila.

También tiene una validación para que no se pueda dividir entre cero y mande un mensaje de error si pasa eso. Al final, el valor que queda en la pila es el resultado de toda la operación.

main.c

```
Code Blame 45 lines (31 loc) · 775 Bytes
1  #include <stdio.h>
2
3  #include "../include/parser.h"
4  #include "../include/evaluator.h"
5  #include "../include/cola.h"
6
7  int main() {
8
9      Variable variables[MAX_VARIABLES];
10
11      int totalVariables;
12
13      char expresion[MAX_EXPR];
14
15      cargarArchivo(
16          "input/casol.txt",
17          variables,
18          &totalVariables,
19          expresion
20      );
21
22      printf("Expresion infija:\n");
23      printf("%s\n\n", expresion);
24
25      Cola* postfija =
26          infijaAPostfija(expresion);
27
28      printf("Expresion postfija:\n");
29
30      imprimirCola(postfija);
31
32      float resultado =
33          evaluarPostfija(
34              postfija,
35              variables,
36              totalVariables
37          );
38
39      printf("\nResultado:\n");
40      printf("%.2f\n", resultado);
41
42      destruirCola(postfija);
43
```

Este archivo se encarga de manejar la cola que usa el programa. Aquí están las funciones para crear la cola, revisar si está vacía, agregar elementos y sacarlos cuando sea necesario.

La cola sirve principalmente para guardar la expresión postfija una vez que ya fue convertida. Como las colas siguen un orden donde el primero que entra es el primero que sale, ayudan bastante a mantener organizada la expresión para después evaluarla correctamente.

También tiene una función para imprimir la cola y ver cómo quedó la expresión ya acomodada, además de otra función que libera la memoria

cuando el programa termina para no dejar cosas ocupando espacio sin necesidad.

parser.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/parser.h"
6  #include "../include/pila.h"
7  #include "../include/cola.h"
8
9  void cargarArchivo(
10     const char* nombreArchivo,
11     Variable variables[],
12     int* totalVariables,
13     char expresion[]
14 ) {
15     FILE *arch = fopen(nombreArchivo, "r");
16     if (arch == NULL) {
17         printf("yo creo que no se va a poder %s.\n", nombreArchivo);
18         return;
19     }
20
21     char linea[MAX_EXPR];
22     *totalVariables = 0;
23
24     while (fgets(linea, sizeof(linea), arch) != NULL) {
25         int longitud = strlen(linea);
26         if (longitud > 0 && linea[longitud - 1] == '\n') {
27             linea[longitud - 1] = '\0';
28         }
29
30         if (strlen(linea) == 0) continue;
31
32         int igual = 0;
33         for (int i = 0; i < strlen(linea); i++) {
34             if (linea[i] == '=') {
35                 igual = 1;
36                 break;
37             }
38         }
39
40         if (igual == 1) {
41             char *nombre = strtok(linea, " =");
42             char *numtext = strtok(NULL, " =");
43         }
44     }
45 }
```



```

44         variables[*totalVariables].nombre = nombre[0];
45         variables[*totalVariables].valor = atof(numtext);
46         *totalVariables = *totalVariables + 1;
47     } else {
48         strcpy(expresion, linea);
49     }
50 }
51
52 fclose(arch);
53 }
54
55 int esOperador(char c) {
56     if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
57         return 1;
58     }
59     return 0;
60 }
61
62 int precedencia(char op) {
63     if (op == '^') {
64         return 3;
65     } else if (op == '*' || op == '/') {
66         return 2;
67     } else if (op == '+' || op == '-') {
68         return 1;
69     }
70     return 0;
71 }
72
73 Cola* infijaAPostfija(char* expresion) {
74     Cola *postfija = crearCola();
75     Pila *pila = crearPila();
76
77     int longitud = strlen(expresion);
78
79     for (int i = 0; i < longitud; i++) {
80         char actual = expresion[i];
81
82         if (actual == ' ') {
83             continue;

```

```

82     if (actual == ' ') {
83         continue;
84     }
85
86     if ((actual >= 'a' && actual <= 'z') || (actual >= 'A' && actual <= 'Z')) {
87         enqueue(postfija, &actual, sizeof(char));
88     }
89     else if (actual == '(') {
90         push(pila, &actual, sizeof(char));
91     }
92     else if (actual == ')') {
93         while (pilaVacía(pila) == 0 && peek(pila) != '(') {
94             char op = pop(pila);
95             enqueue(postfija, &op, sizeof(char));
96         }
97         if (pilaVacía(pila) == 0) {
98             pop(pila);
99         }
100     }
101     else if (esOperador(actual) == 1) {
102         while (pilaVacía(pila) == 0 && peek(pila) != '(') {
103             char tope = peek(pila);
104             int ant = precedencia(actual);
105             int precTope = precedencia(tope);
106
107             int sacar = 0;
108
109             if (actual != '^' && precTope >= ant) {
110                 sacar = 1;
111             } else if (actual == '^' && precTope > ant) {
112                 sacar = 1;
113             }
114
115             if (sacar == 1) {
116                 char op = pop(pila);
117                 enqueue(postfija, &op, sizeof(char));
118             } else {
119                 break;
120             }

```

```

120         }
121     }
122     push(pila, &actual, sizeof(char));
123 }
124 }
125
126 while (pilaVacía(pila) == 0) {
127     char op = pop(pila);
128     enqueue(postfija, &op, sizeof(char));
129 }
130
131 destruirPila(pila);
132 return postfija;
133 }

```

Este archivo es donde se hace gran parte del trabajo importante del programa. Primero se encarga de abrir el archivo de texto y leer toda la información. Si encuentra variables, guarda su nombre y su valor; si encuentra la operación matemática, la guarda para trabajar con ella después.

Además, aquí están las funciones que ayudan a identificar $+$, $-$, $*$, además de decidir cuáles tienen mayor prioridad. Esto es importante porque no todas las operaciones se hacen en el mismo orden.

La parte más importante de este archivo es cuando la expresión infija se convierte a postfija. Para eso se usa una pila, donde temporalmente se guardan operadores y paréntesis para acomodarlos correctamente. Gracias a eso, el programa puede respetar el orden de las operaciones sin que se haga un desorden al momento de resolverlas.

pila.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/pila.h"
6
7  Pila* crearPila() {
8      Pila *p = (Pila *)malloc(sizeof(Pila));
9      if (p == NULL) {
10         printf("Error: sin memoria para crear pila.\n");
11         return NULL;
12     }
13     p->tope = NULL;
14     return p;
15 }
16
17 // Se reserva memoria para el TDA "pila" y se verifica que no este vacia, si no lo esta, se asigna NULL al tope
18
19 int pilaVacia(Pila *p) {
20     return p->tope == NULL;
21 }
22
23 void push(Pila *p, void *dato, size_t size) {
24     NodoPila *nuevo = (NodoPila *)malloc(sizeof(NodoPila));
25     if (nuevo == NULL) {
26         printf("Error: sin memoria para push.\n");
27         return;
28     }
29     nuevo->dato = malloc(size);
30     if (nuevo->dato == NULL) {
31         printf("Error: sin memoria para dato en push.\n");
32         return;
33     }
34     memcpy(nuevo->dato, dato, size);
35     nuevo->siguiente = p->tope;
36     p->tope = nuevo;
37 }
38
39 // Ingresa un nuevo elemento en la cima de la pila
40
41 char pop(Pila *p) {
42     if (pilaVacia(p)) {
43         printf("Error: pop en pila vacia.\n");
44         return '\0';
45     }
46 }
```

```

44     return '\0';
45 }
46 NodoPila *temp = p->tope;
47 p->tope = temp->siguiente;
48 char dato = *((char *)temp->dato);
49 free(temp->dato);
50 free(temp);
51 return dato;
52 }
53
54 char peek(Pila *p) {
55     if (pilaVacía(p)) return '\0';
56     return *((char *)p->tope->dato);
57 }
58
59 void destruirPila(Pila *p) {
60     while (!pilaVacía(p)) {
61         NodoPila *temp = p->tope;
62         p->tope = temp->siguiente;
63         free(temp->dato);
64         free(temp);
65     }
66     free(p);
67 }
68

```

Este archivo se encarga de manejar la pila que usa el programa durante la conversión de la expresión. La pila funciona de forma sencilla: el último dato que entra es el primero que sale, algo parecido a una pila de cuadernos donde quitas primero el de arriba.

Aquí están las funciones para crear la pila, verificar si tiene datos, meter elementos (push), sacar elementos (pop). Estas funciones ayudan a que el programa pueda organizar temporalmente los operadores mientras convierte la expresión.

La pila es importante porque permite respetar el orden correcto de las operaciones matemáticas y también ayuda cuando hay paréntesis, evitando que algo salga mal en la conversión. Al terminar, la memoria usada se libera para que el programa no tenga problemas después.

Conclusión

En conclusión, este proyecto sirvió para entender mejor cómo se pueden resolver expresiones matemáticas usando programación en C, ya que no solo fue hacer operaciones, también ayudó a comprender mejor cómo organizar un programa para que cada parte haga una función diferente y no esté todo mezclado, además se puso en práctica el uso de pilas y colas, que al principio pueden parecer algo confusas pero durante el proyecto ayudaron bastante para mantener el orden correcto de las operaciones y poder convertir expresiones de infijas a postfijas sin errores, también fue útil trabajar con archivos de texto para leer variables y expresiones automáticamente en lugar de escribir todo manualmente, lo que hizo el programa un poco más completo, además de eso se reforzaron temas como el uso de memoria dinámica, funciones, estructuras y la organización del

código en varios archivos para que fuera más fácil de entender y corregir si algo salía mal, en general fue una práctica que ayudó bastante a ver cómo muchas cosas que se ven por separado en clase realmente se juntan para hacer un programa que sí funciona y resuelve algo de forma más ordenada.